

GUI Agent with Local LLM/VLM and OpenVINO

Building a privacy-first, offline-capable desktop automation agent powered by locally deployed vision-language models

Contributor: T.D. Dervish | GitHub: [tddervish-max](https://github.com/tddervish-max) | <https://github.com/tddervish-max>

Mentors: Ethan Yang, Zhuo Wu | Size: 350 hours | Difficulty: Hard

PROPOSAL TITLE	GUI Agent with Local LLM/VLM & OpenVINO	350 hours (Large / Hard)	tddervish-max
----------------	---	--------------------------	--

github.com/tddervish-max

Python	OpenVINO	LLM / VLM	PyAutoGUI	Tkinter / PyQt
Prompt Engineering	Agentic Workflow	ONNX	Computer Vision	Desktop Automation

1. Proposal Title

OpenVINO GUI Agent: A Privacy-First Desktop Automation System Powered by Locally Deployed Vision-Language Models

2. Proposal Summary

Modern GUI agents — systems that can autonomously operate a computer screen in response to natural language instructions — have traditionally depended on cloud-hosted large language models. This creates serious barriers: latency, cost, privacy risk, and the requirement for a constant internet connection. This project solves all four problems by building a fully local, open-source GUI agent desktop application where every AI model runs on the user's own hardware, accelerated by Intel OpenVINO.

The agent will accept high-level user instructions (e.g., *'Open Firefox, search for flight prices to Paris, and paste the cheapest result into my spreadsheet'*) and autonomously decompose them into a sequence of low-level GUI actions — mouse clicks, keyboard input, window management — executing them in a reliable loop with perception, planning, action, and reflection steps.

The core pipeline will be: **(1) Screen Perception** — a locally deployed Vision-Language Model (VLM) such as a quantized MiniCPM-V or Qwen-VL, optimized with OpenVINO, reads the current screen state; **(2) Planning** — a locally deployed LLM (e.g., Phi-3 or Mistral, also OpenVINO-optimized) decomposes the user goal into atomic steps; **(3) Action Execution** — PyAutoGUI or pywinauto executes the predicted action; **(4) Reflection** — the VLM verifies the result and decides whether to continue, retry, or report completion. The desktop GUI wrapper (PyQt6 or Tauri) gives the user a clean interface to issue commands, monitor progress, and intervene if needed.

Inspired by the architectures of UI-TARS-desktop and MobileAgent, this project adapts their agent loop designs for a local-first, OpenVINO-accelerated deployment, making advanced GUI automation accessible without cloud dependencies.

3. Problem Statement & Motivation

Existing GUI automation tools (RPA, Selenium, PyAutoGUI scripts) require hardcoded selectors or pixel coordinates — they are brittle, maintenance-heavy, and cannot generalize to new applications. AI-powered GUI agents (like those from Anthropic's Computer Use or Microsoft's Copilot) are cloud-dependent, raising enterprise privacy concerns and making offline deployment impossible.

This project addresses three open problems:

- **Privacy:** all inference happens on-device — no user screen data leaves the machine.
- **Performance:** OpenVINO's INT8/INT4 quantization and hardware-aware kernel fusion deliver GPU/NPU-level throughput on standard AIPC hardware without requiring NVIDIA GPUs.
- **Generalization:** VLM-based screen understanding means the agent works across any application — browser, IDE, spreadsheet, native app — without bespoke automation code.

4. Technical Architecture

4.1 System Overview

The application is structured as four loosely coupled modules communicating via a shared state object. This architecture mirrors UI-TARS's approach but replaces cloud API calls with local OpenVINO inference engines.

Module	Responsibility	Key Technology
Screen Capture & OCR	Capture current screen; extract UI elements and text regions	textesseract, pywinauto
VLM Perception Engine	Understand screen state visually; identify interactive elements	Qwen-VL / Qwen-VL via OpenVINO
LLM Planning Engine	Decompose user goal into ordered action steps; handle re-planning	Phi-3-mini / Mistral via OpenVINO
Action Executor	Translate planned actions to OS-level input events	PyAutoGUI, pywinauto, keyboard lib
GUI Frontend	User instruction input; live action log; pause/stop controls	PyQt6 / Tkinter
Reflection Loop	Post-action verification; error recovery; completion detection	ML verification + rule-based checks

4.2 OpenVINO Model Pipeline

Both the VLM and LLM will be exported to OpenVINO IR format using optimum-intel and optimized with NNCF for INT8 weight compression, targeting Intel Core Ultra (Meteor Lake) NPU + iGPU. The inference pipeline:

```
# Model export & optimization (one-time setup)
from optimum.intel import OVModelForCausalLM
model = OVModelForCausalLM.from_pretrained('microsoft/Phi-3-mini-4k-instruct',
    export=True, load_in_8bit=True)
model.save_pretrained('./phi3-ov-int8')

# Runtime inference
from openvino.runtime import Core
core = Core()
```

```
compiled = core.compile_model('./phi3-ov-int8/opencvino_model.xml', 'NPU')
```

4.3 Agent Loop

The core agentic loop follows an observe-plan-act-reflect cycle:

```
def agent_loop(user_instruction):
    history = []
    for step in range(MAX_STEPS):
        screen = capture_screen() # mss screenshot
        state = vlm_perceive(screen) # OpenVINO VLM
        action = llm_plan(user_instruction, # OpenVINO LLM
                          state, history)
        execute_action(action) # PyAutoGUI
        if vlm_reflect(screen, action): break # goal met?
```

4.4 GUI Frontend

The desktop application will be built with PyQt6, providing: a natural language instruction input bar, a live screen preview panel with bounding-box overlays showing where the agent is looking, a scrollable action log, and pause/abort/replay controls. A settings panel will allow users to select model paths, target device (CPU/GPU/NPU), and inference precision.

5. Required Technologies

Category	Technology	Role in Project
AI Runtime	Intel OpenVINO 2024+	Model optimization, INT8 quantization, NPU/GPU dispatch
AI Runtime	optimum-intel / NNCF	HuggingFace model export to OpenVINO IR + weight compression
Vision-Language	MiniCPM-V 2.6 / Qwen-VL-Chat	Screen state perception; UI element understanding
Language Model	Phi-3-mini / Mistral-7B	Instruction decomposition; action planning; reflection
GUI Automation	PyAutoGUI	Cross-platform mouse/keyboard event generation
GUI Automation	pywinauto	Windows accessibility tree traversal; reliable element click
Screen Capture	mss	Fast cross-platform screen capture at ~60fps
OCR	pytesseract / easyocr	Text extraction from screen regions without UI hooks
Frontend	PyQt6	Desktop application GUI; live preview; action log
Prompt Eng.	LangChain / custom templates	Structured prompting; few-shot examples; chain-of-thought
Dev / Testing	pytest, GitHub Actions	Unit tests, integration tests, CI pipeline
Language	Python 3.11+	Primary implementation language

6. Project Timeline (350 hours)

Phase	Period	Deliverables	Hrs
Community Bonding	May 1–26	Study UI-TARS & MobileAgent codebases; benchmark OpenVINO model exports for Phi-3 and MiniCPM-V; a	20
Phase 1 Foundation	May 27– Jun 16	OpenVINO inference wrappers for LLM + VLM; screen capture module; basic action executor (click, type, scr	70
Phase 2 Agent Loop	Jun 17– Jul 7	Full observe-plan-act-reflect loop; prompt templates for planning and reflection; history/memory management	70
Phase 3 GUI Frontend	Jul 8– Jul 28	PyQt6 desktop application; instruction input; live screen preview with overlays; action log panel; pause/stop/r	60
Phase 4 Robustness	Jul 29– Aug 11	Cross-application testing (browser, file manager, spreadsheet); edge case handling; pywinauto fallback for ac	50
Phase 5 NPU Optimization	Aug 12– Aug 22	INT4 quantization experiments; NPU dispatch tuning; benchmark report (latency/accuracy) on AIPC cloud ha	40
Phase 6 Docs & Release	Aug 23– Sep 1	README, installation guide, video demo, API docs; final PR; address all mentor review comments; submit fir	40
Buffer	Throughout	Mentor feedback cycles, unexpected complexity, cross-platform compatibility fixes	50

7. Expected Outcomes & Deliverables

- A fully installable desktop application (Windows + Linux) providing a native GUI Agent powered entirely by local models.
- OpenVINO-optimized inference pipeline for at least one VLM (screen perception) and one LLM (planning), with INT8 compression.
- Reliable agentic loop: observe → plan → act → reflect, with configurable max-step limits and automatic error recovery.
- PyQt6 GUI with live screen preview, bounding-box overlays, action log, and user control panel.
- Benchmark report comparing NPU vs CPU vs GPU performance on AIPC cloud hardware.
- Comprehensive test suite (unit + integration) with CI via GitHub Actions.
- Full documentation: README, installation guide, architecture diagram, and a demo video.
- All code released as open source under Apache 2.0, merged via reviewed PRs.

8. About the Contributor

Name	T.D. Dervish	GitHub	tddervish-max
Profile URL	https://github.com/tddervish-max	Experience	Expert — OpenVINO / Production AI Systems

I have expert-level experience deploying AI systems in production with OpenVINO, including quantizing transformer models for edge and AIPC hardware. I have hands-on experience with GUI automation frameworks (PyAutoGUI, pywinauto) and have built pipelines that combine screen capture, OCR, and programmatic UI control.

I have studied the UI-TARS-desktop and MobileAgent repositories in depth and understand their agent loop architectures, prompt engineering strategies, and action space designs. I am well-positioned to adapt and improve

on these for a local-first, OpenVINO-accelerated deployment.

Availability: I will commit approximately 35–40 hours per week throughout the GSoC coding period, with no major competing obligations. I will maintain weekly written updates to mentors, keep a public development branch, and proactively flag blockers early.

9. References & Inspiration

- UI-TARS Desktop — <https://github.com/bytedance/UI-TARS-desktop>
- MobileAgent — <https://github.com/X-PLUG/MobileAgent>
- Intel OpenVINO Toolkit — <https://github.com/openvinotoolkit/openvino>
- optimum-intel (HuggingFace x OpenVINO) — <https://github.com/huggingface/optimum-intel>
- MiniCPM-V Vision-Language Model — <https://github.com/OpenBMB/MiniCPM-V>
- PyAutoGUI — <https://pyautogui.readthedocs.io>
- pywinauto — <https://pywinauto.readthedocs.io>